

5 - MAC: AppArmor & SELinux

Name of report: AppArmSELinux_LAB_5_Nikita_Niakhai

Course: Secure Development

Performed by *Nikita Niakhai*

Date submission: 03.04.2026

Theory rehearsal

MAC

Key characteristics:

- Centralized control: only admin(s) set rules and control the access policy, no one else can alter and change it. System defines rules.
- No user discretion: user can not change and share access to his resources. He is a unit in the system without any opportunity to effect access policy.
- Policy based: rules in the system defined by formal policies.
- High Security assurance: we are confident that system is not misconfigured and there are no data breaches, everything is setup precisely without any bugs

Types of MAC:

- multi level: simple hierarchy
- multi lateral: many sides, each side has its own level.

Working of MAC:

- signal to check right
- waking up MAC to evaluate clearance and classification and compare them

Policy:

Each object can be defined into a group of objects

Each level access (clearance) has his access rules to each group of objects

Clearance (con, context) — is a security context that each process/user has. Basically a user.

Classification — . Basically a resource we want to access.

Practical examples:

`ls -Z object_path` - used to check MAC's clearance, gives output:

`[user]:[role]:[type]:[level]`

- `type` = classification (e.g., `user_home_t`)
- `level` = sensitivity (e.g., `s0`)

Used in MAC implementations such as in `SELinux`, `AppArmor`. There is a mechanism that checks files/directories classification (labels) and compares with users clearance, if they do not match → no access. Creates access control that works on the top of UNIX `chmod`, `chown`.

Mandatory Access Control - GeeksforGeeks

Your All-in-One Learning Portal: GeeksforGeeks is a comprehensive educational platform that empowers learners across domains-spanning computer science and programming, <https://www.geeksforgeeks.org/ethical-hacking/mandatory-access-control/>



Centralized Control



Policy-Based Access



No User Discretion



High Security Assurance

PART A – AppArmor

A1 – CIS Benchmarks: How They Are Checked on an Endpoint

CIS benchmarks are checked using automated compliance scanners that audit system settings against published CIS controls. In a SIEM context (e.g. Wazuh), the agent runs CIS-CAT or the built-in SCA (Security Configuration Assessment) module, comparing current settings to expected benchmark values and reporting pass/fail results to the SIEM dashboard.

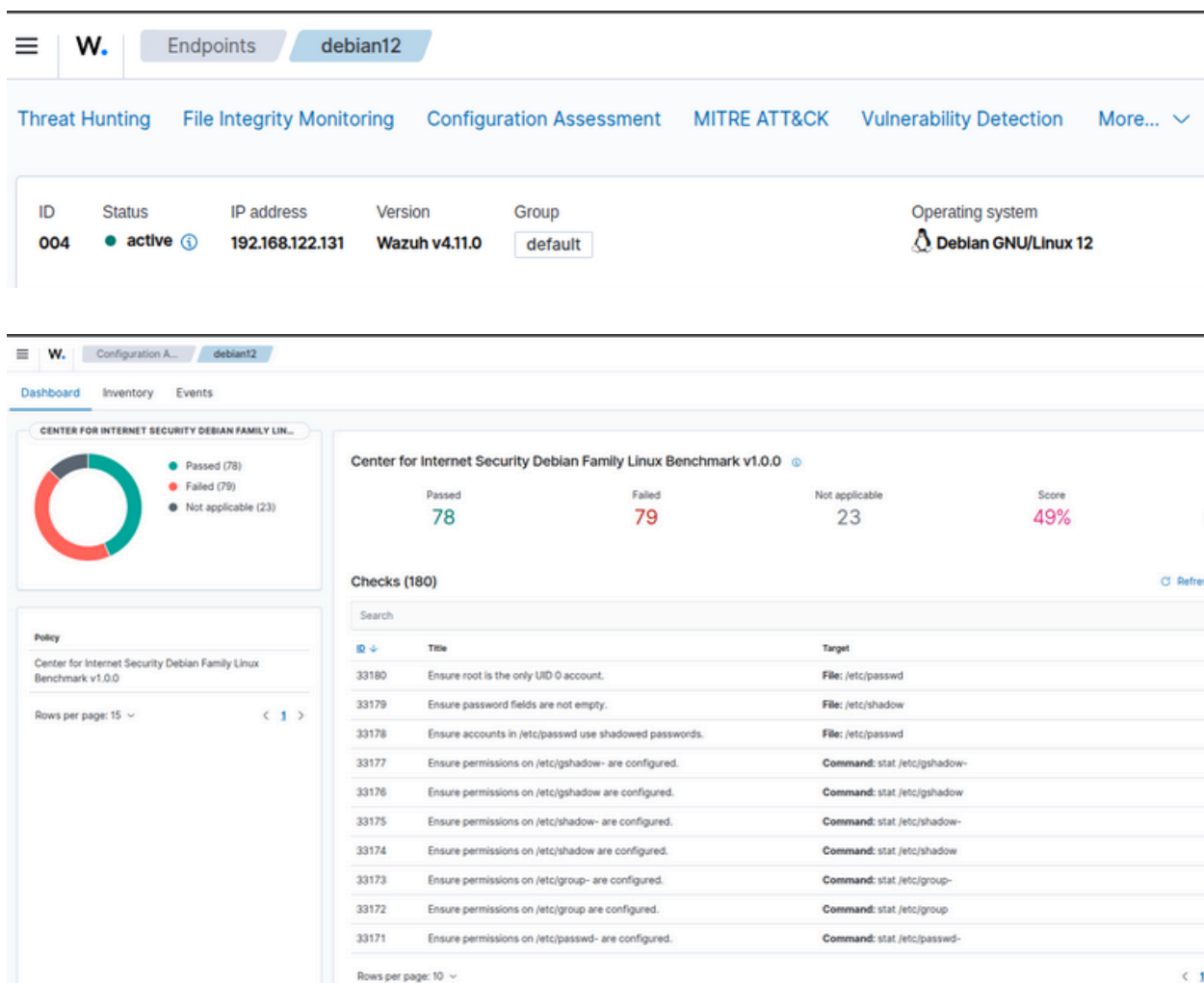


Figure. Figure CIS benchmarks are checked

SCA policy checks could be enabled and configured from manager config [\[ref.\]](#)

A2 – CIS Benchmark: MAC Section (AppArmor Controls)

Linux Ubuntu 22.04

1.6.1 – Ensure AppArmor is installed

```
sudo apt install -y apparmor apparmor-utils apparmor-profiles
apparmor-profiles-extra
```

1.6.1.1 – Ensure AppArmor is enabled in the bootloader

```
# Check current GRUB config
grep "GRUB_CMDLINE_LINUX" /etc/default/grub

# Add apparmor=1 and security=apparmor to GRUB
sudo sed -i 's/GRUB_CMDLINE_LINUX="\(.*\)"/GRUB_CMDLINE_LINUX
="\1 apparmor=1 security=apparmor"/' \
/etc/default/grub

# Verify the change
grep "GRUB_CMDLINE_LINUX" /etc/default/grub

# Update GRUB
sudo update-grub

# Reboot to apply kernel parameters
sudo reboot
```

```

root@UbuntuDesktop:~# grep "GRUB_CMDLINE_LINUX" /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX=""
root@UbuntuDesktop:~# sudo sed -i 's/GRUB_CMDLINE_LINUX="(.*\)" /GRUB_CMDLINE_LINUX
="1 apparmor=1 security=apparmor"/' /etc/default/grub
root@UbuntuDesktop:~# grep "GRUB_CMDLINE_LINUX" /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX=" apparmor=1 security=apparmor"
root@UbuntuDesktop:~# sudo update-grub
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-20-generic
Found initrd image: /boot/initrd.img-6.17.0-20-generic
Found linux image: /boot/vmlinuz-6.14.0-37-generic
Found initrd image: /boot/initrd.img-6.14.0-37-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done

```

Figure. AppArmor is verified to be installed at grub level

After reboot – Verify AppArmor is active

```

# Check AppArmor status
sudo apparmor_status

# Or via systemd
sudo systemctl status apparmor

# Verify kernel parameter
cat /proc/cmdline | grep apparmor

```

```

0 processes are in enforce mode.
root@UbuntuDesktop:~# systemctl status apparmor
● apparmor.service - Load AppArmor profiles
   Loaded: loaded (/usr/lib/systemd/system/apparmor.service; enabled; preset: enable>
   Active: active (exited) since Thu 2026-04-02 18:59:23 UTC; 2min 26s ago
     Docs: man:apparmor(7)
           https://gitlab.com/apparmor/apparmor/wikis/home/
   Main PID: 332 (code=exited, status=0/SUCCESS)
      CPU: 6.546s

Apr 02 18:58:20 UbuntuDesktop systemd[1]: Starting apparmor.service - Load AppArmor pr>
Apr 02 18:58:20 UbuntuDesktop apparmor.systemd[332]: Restarting AppArmor
Apr 02 18:58:20 UbuntuDesktop apparmor.systemd[332]: Reloading AppArmor profiles
Apr 02 18:59:23 UbuntuDesktop apparmor.systemd[798]: Warning: found usr.sbin.sssd in />
Apr 02 18:59:23 UbuntuDesktop apparmor.systemd[798]: Warning from /etc/apparmor.d (/et>
Apr 02 18:59:23 UbuntuDesktop systemd[1]: Finished apparmor.service - Load AppArmor pr>
lines 1-14/14 (END)
cat ^C
root@UbuntuDesktop:~# cat /proc/cmdline | grep apparmor
BOOT_IMAGE=/boot/vmlinuz-6.17.0-20-generic root=UUID=5ed2a28f-d214-4b01-81c9-1b09507084
03 ro apparmor=1 security=apparmor quiet splash
root@UbuntuDesktop:~#

```

```

root@UbuntuDesktop:~# sudo apparmor_status
apparmor module is loaded.
187 profiles are loaded.
89 profiles are in enforce mode.

```

Figure. AppArmor is active and profiles are loaded

1.6.1.2 – Ensure all AppArmor profiles are in enforce mode

```

# Check current profile status
sudo aa-status

# Set all loaded profiles to enforce mode
sudo aa-enforce /etc/apparmor.d/*

```

```

root@UbuntuDesktop:~# aa-enforce /etc/apparmor.d/*
Setting /etc/apparmor.d/1password to enforce mode.
Profile for /etc/apparmor.d/abi not found, skipping
Profile for /etc/apparmor.d/abstractions not found, skipping
Profile for /etc/apparmor.d/apache2.d not found, skipping
Setting /etc/apparmor.d/balena-etcher to enforce mode.
Setting /etc/apparmor.d/bin.ping to enforce mode.

```

Figure. Enforcing profiles

```
# Load all available profiles
sudo apparmor_parser -r /etc/apparmor.d/*

# Verify: no profiles should be in complain mode
sudo aa-status | grep "profiles are in complain mode"
# Should return: 0 profiles are in complain mode

# Check for unconfined processes
sudo aa-status | grep "processes are unconfined"
```

```
root@UbuntuDesktop:~# apparmor_parser -r /etc/apparmor.d/*
AppArmor parser error for /etc/apparmor.d/abi in profile /etc/apparmor.d/abi/3.0 at line 1: syntax error, unexpected TOK_OPEN, expecting TOK_MODE
AppArmor parser error for /etc/apparmor.d/abi in profile /etc/apparmor.d/abi/4.0 at line 1: syntax error, unexpected TOK_CAPABILITY, expecting end of file
AppArmor parser error for /etc/apparmor.d/abi in profile /etc/apparmor.d/abi/kernel-5.4: outoftree-network at line 1: syntax error, unexpected TOK_OPEN, expecting TOK_MODE
AppArmor parser error for /etc/apparmor.d/abi in profile /etc/apparmor.d/abi/kernel-5.4
```

Figure. Loading all available profiles

```
Found reference to variable run, but is never declared
root@UbuntuDesktop:~# aa-status | grep "profiles are in complain mode"
0 profiles are in complain mode.
root@UbuntuDesktop:~# aa-status | grep "processes are unconfined"
3 processes are unconfined but have a profile defined.
root@UbuntuDesktop:~#
```

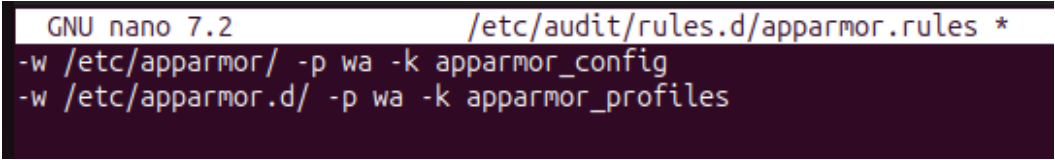
Figure. Verifying complain mode for profiles and unconfined processes

Additional Hardening (CIS-aligned)

```
# Install audit daemon for AppArmor logging
sudo apt install -y auditd audispd-plugins

sudo systemctl enable auditd
sudo systemctl start auditd
```

```
# Configure auditd for AppArmor events
sudo tee -a /etc/audit/rules.d/apparmor.rules << 'EOF'
## AppArmor audit rules
-w /etc/apparmor/ -p wa -k apparmor_config
-w /etc/apparmor.d/ -p wa -k apparmor_profiles
EOF
```



```
GNU nano 7.2 /etc/audit/rules.d/apparmor.rules *
-w /etc/apparmor/ -p wa -k apparmor_config
-w /etc/apparmor.d/ -p wa -k apparmor_profiles
```

Figure. `auditd` for AppArmor events

```
sudo service auditd restart

# Verify AppArmor kernel module is loaded
sudo cat /sys/module/apparmor/parameters/enabled
# Should return: Y

# Check AppArmor filesystem
ls /sys/kernel/security/apparmor/

# List all profiles and their modes
sudo aa-status --json | python3 -m json.tool
```



```

root@UbuntuDesktop:~# service auditd restart
root@UbuntuDesktop:~# cat /sys/module/apparmor/parameters/enabled
Y
root@UbuntuDesktop:~# ls /sys/kernel/security/apparmor/
features  profiles                                raw_data_compression_level_min
policy    raw_data_compression_level_max  revision
root@UbuntuDesktop:~# sudo aa-status --json | python3 -m json.tool
{
  "version": "2",
  "profiles": {
    "/snap/snapd/24792/usr/lib/snapd/snap-confine": "enforce",
    "/snap/snapd/24792/usr/lib/snapd/snap-confine//mount-namespace-capture-helper":
"enforce",
    "/usr/bin/evince": "enforce",
    "/usr/bin/evince-previewer": "enforce",
    "/usr/bin/evince-previewer//sanitized_helper": "enforce",
    "/usr/bin/evince-thumbnailer": "enforce",

```

Figure. `auditd` restarted, profiles and their model listed and verified manually

Ensure AppArmor starts on boot

```

sudo systemctl enable apparmor
sudo systemctl is-enabled apparmor
# Should output: enabled

```

```

}
root@UbuntuDesktop:~# systemctl enable apparmor
Synchronizing state of apparmor.service with SysV service script with /usr/lib/systemd/
systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apparmor
root@UbuntuDesktop:~# systemctl is-enabled apparmor
enabled
root@UbuntuDesktop:~#

```

Figure. Reloading AppArmor

A3 – Webapp Serving Two Directories with AppArmor Confinement

This section deploys Nginx to serve files from two directories (`/var/www/allowed` and `/var/www/restricted`), then uses AppArmor to confine Nginx to only one directory.

Step 1 – Install Nginx

```
sudo apt install -y nginx
```

```
# Enable and start nginx
```

```
sudo systemctl enable nginx
```

```
sudo systemctl start nginx
```

```
sudo systemctl status nginx
```

```
root@UbuntuDesktop:~# systemctl start nginx
root@UbuntuDesktop:~# systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-04-02 19:46:58 UTC; 23s ago
     Docs: man:nginx(8)
   Main PID: 5338 (nginx)
    Tasks: 3 (limit: 7159)
   Memory: 2.4M (peak: 5.3M)
      CPU: 34ms
   CGroup: /system.slice/nginx.service
           └─5338 "nginx: master process /usr/sbin/nginx -g daemon on; master_process"
              └─5339 "nginx: worker process"
                 └─5340 "nginx: worker process"

Apr 02 19:46:58 UbuntuDesktop systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server: >
Apr 02 19:46:58 UbuntuDesktop systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server: >
```

Figure. Nginx is installed

Step 2 – Create two content directories

```
# Create directories
```

```
sudo mkdir -p /var/www/allowed
```

```
sudo mkdir -p /var/www/restricted
```

```
# Create test files in each directory
```

```
echo "<h1>ALLOWED DIRECTORY - This file is publicly accessible</h1>" \
```

```
| sudo tee /var/www/allowed/index.html
```

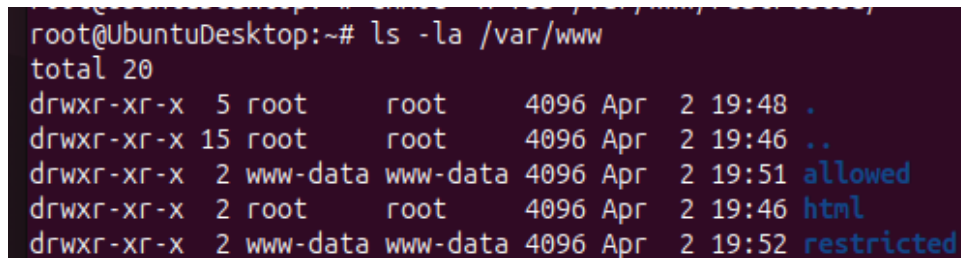
```
echo "<h1>RESTRICTED DIRECTORY - This should be blocked by AppArmor</h1>" \
```

```
| sudo tee /var/www/restricted/secret.html

# Set ownership
sudo chown -R www-data:www-data /var/www/allowed
sudo chown -R www-data:www-data /var/www/restricted

# Set permissions
sudo chmod -R 755 /var/www/allowed
sudo chmod -R 755 /var/www/restricted

# Verify
ls -la /var/www/allowed/
ls -la /var/www/restricted/
```



```
root@UbuntuDesktop:~# ls -la /var/www
total 20
drwxr-xr-x  5 root    root    4096 Apr  2 19:48 .
drwxr-xr-x 15 root    root    4096 Apr  2 19:46 ..
drwxr-xr-x  2 www-data www-data 4096 Apr  2 19:51 allowed
drwxr-xr-x  2 root    root    4096 Apr  2 19:46 html
drwxr-xr-x  2 www-data www-data 4096 Apr  2 19:52 restricted
```

Figure. Ownership and Permissions for web files

Step 3 – Configure Nginx to serve both directories

```
# Create Nginx configuration
sudo tee /etc/nginx/sites-available/lab5 << 'EOF'
server {
    listen 80;
    server_name _;

    # Serve the allowed directory at /allowed
    location /allowed/ {
        alias /var/www/allowed/;
        autoindex on;
    }
}
```

```

# Serve the restricted directory at /restricted
location /restricted/ {
    alias /var/www/restricted/;
    autoindex on;
}

# Default location
location / {
    return 200 "Lab5 AppArmor Demo Server\n";
    add_header Content-Type text/plain;
}
}
EOF

# Enable the site and disable the default
sudo ln -sf /etc/nginx/sites-available/lab5 /etc/nginx/sites-enabled/lab5
sudo rm -f /etc/nginx/sites-enabled/default

# Test Nginx configuration
sudo nginx -t

# Reload Nginx
sudo systemctl reload nginx

```

Step 4 – Verify BOTH directories are accessible (AppArmor INACTIVE)

```

# Get server IP
SERVER_IP=$(hostname -I | awk '{print $1}')
echo "Server IP:$SERVER_IP"

# Test allowed directory
curl -v http://localhost/allowed/index.html

```

```
# Expected: 200 OK with "ALLOWED DIRECTORY" content

# Test restricted directory
curl -v http://localhost/restricted/secret.html
# Expected: 200 OK with "RESTRICTED DIRECTORY" content (AppArmor not yet active)

# Check AppArmor status for nginx
sudo aa-status | grep nginx
```

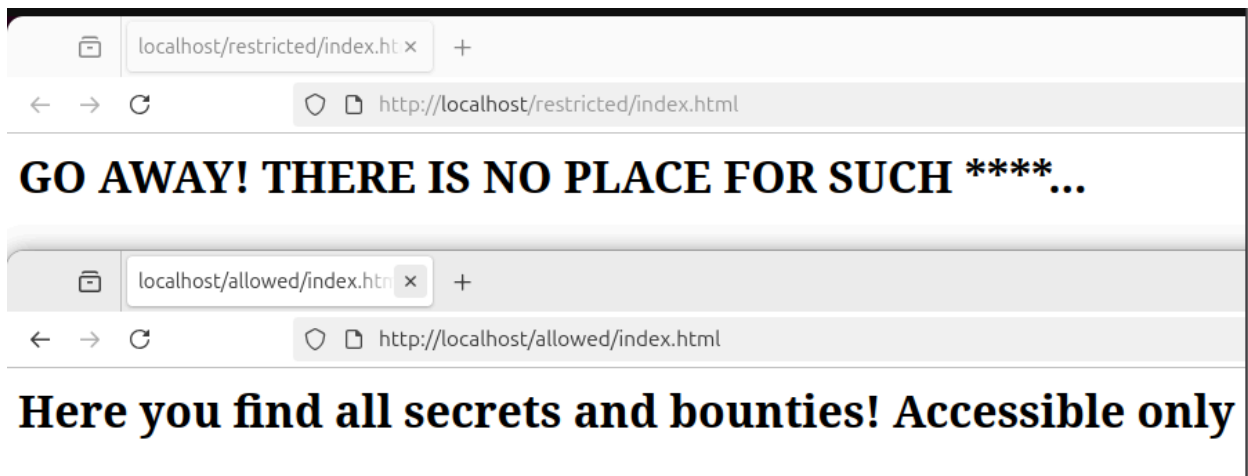


Figure. Accessing web directories

```
root@UbuntuDesktop:/media# aa-status | grep nginx
root@UbuntuDesktop:/media# cat /etc/apparmor.d/usr.bin.nginx
cat: /etc/apparmor.d/usr.bin.nginx: No such file or directory
```

Figure. Preparing AppArmor for nginx profile

Step 5 – Check the existing Nginx AppArmor profile

```
# View the default nginx profile
sudo cat /etc/apparmor.d/usr.sbin.nginx

# Or if not present, install extra profiles
sudo apt install -y apparmor-profiles-extra
```

```
# List all available profiles
ls /etc/apparmor.d/ | grep -i nginx
```

- No profile for `nginx` by default, so let's create one

Step 6 – Create a custom AppArmor profile for Nginx

```
# First, put the existing nginx profile in complain mode to see what it accesses
sudo aa-complain /usr/sbin/nginx 2>/dev/null || true
```

```
# Now create a custom restrictive profile
sudo tee /etc/apparmor.d/usr.sbin.nginx << 'PROFILE'
#include <tunables/global>
```

```
/usr/sbin/nginx {
    #include <abstractions/base>
    #include <abstractions/nameservice>
    #include <abstractions/openssl>
```

```
    capability dac_override,
    capability net_bind_service,
    capability setgid,
    capability setuid,
```

```
    # Binary
    /usr/sbin/nginx mr,
```

```
    # Configuration files
    /etc/nginx/** r,
    /etc/ssl/** r,
```

```
    # PID and lock files
    /run/nginx.pid rw,
    /var/lock/nginx.lock rw,
```

```

# Logs
/var/log/nginx/ rw,
/var/log/nginx/** rw,

# Temporary files
/var/lib/nginx/ rw,
/var/lib/nginx/** rw,
/tmp/ rw,
/tmp/** rw,

# ALLOWED DIRECTORY - Nginx CAN read this
/var/www/allowed/ r,
/var/www/allowed/** r,

# RESTRICTED DIRECTORY - intentionally OMITTED
# /var/www/restricted/ r, <-- NOT ALLOWED
# /var/www/restricted/** r, <-- NOT ALLOWED

# System libraries
/lib/** mr,
/usr/lib/** mr,
/usr/share/nginx/** r,

# Network
network tcp,
network udp,
}
PROFILE

echo "Custom AppArmor profile created."

```

```

network udp,
}
Custom AppArmor profile created.
root@UbuntuDesktop:/media#

```

Figure. Profile for `nginx` is created

Step 7 – Load and enforce the Nginx AppArmor profile

```
# Parse and load the profile
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.nginx

# Set to enforce mode
sudo aa-enforce /usr/sbin/nginx

# Verify profile is loaded and enforcing
sudo aa-status | grep -A2 nginx

# Restart Nginx so the new confinement applies
sudo systemctl restart nginx
sudo systemctl status nginx
```

```
root@UbuntuDesktop:/media# apparmor_parser -r /etc/apparmor.d/usr.sbin.nginx
root@UbuntuDesktop:/media# aa-enforce /usr/sbin/nginx
Setting /usr/sbin/nginx to enforce mode.
root@UbuntuDesktop:/media# ss-status | grep nginx
ss-status: command not found
root@UbuntuDesktop:/media# aa-status | grep nginx
/usr/sbin/nginx
/usr/sbin/nginx (5338)
/usr/sbin/nginx (9400)
/usr/sbin/nginx (9401)
root@UbuntuDesktop:/media# systemctl restart nginx
root@UbuntuDesktop:/media# systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset:➤
   Active: active (running) since Thu 2026-04-02 20:07:08 UTC; 4s ago
     Docs: man:nginx(8)
   Process: 12374 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_p>
   Process: 12380 ExecStart=/usr/sbin/nginx -g daemon on; master_process on>
  Main PID: 12381 (nginx)
    Tasks: 3 (limit: 7159)
   Memory: 2.4M (peak: 2.6M)
      CPU: 18ms
   CGroup: /system.slice/nginx.service
```


Figure. Reloading, parsing, applying profiles and services

Step 8 – Verify directory access WITH AppArmor ENFORCING

```
# Test allowed directory - SHOULD SUCCEED
echo "=== Testing ALLOWED directory (should work) ==="
curl -v http://localhost/allowed/index.html
echo ""

# Test restricted directory - SHOULD FAIL (403 or empty)
echo "=== Testing RESTRICTED directory (should be blocked) ==
=="
curl -v http://localhost/restricted/secret.html
echo ""

# Check AppArmor denial logs
sudo dmesg | grep -i "apparmor.*DENIED" | tail -20
# OR
sudo tail -20 /var/log/syslog | grep -i apparmor
# OR
sudo ausearch -m avc --start today 2>/dev/null | grep nginx
```

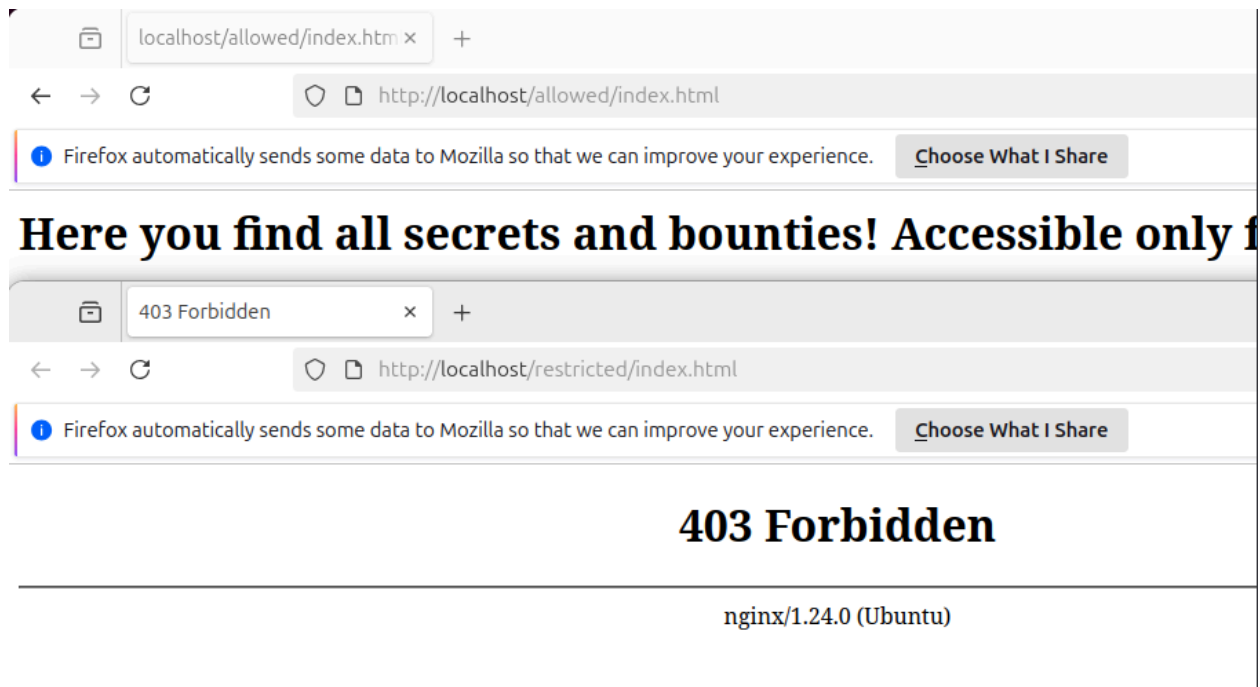


Figure. Hardening and MAC applied

```
root@UbuntuDesktop:/media# dmesg | grep -i "apparmor.*DENIED*" | tail -20
[ 19.318381] audit: type=1400 audit(1775156372.296:194): apparmor="DENIED" operation="capable"
class="cap" profile="/usr/sbin/cupsd" pid=1100 comm="cupsd" capability=12 capname="net admin"
```

Figure.

Step 9 – External access test (from another machine or using port-forwarding)

```
# Allow HTTP through firewall
sudo ufw allow 80/tcp
sudo ufw enable

# From a remote client, test:
# curl http://<SERVER_IP>/allowed/index.html → Should succeed
# curl http://<SERVER_IP>/restricted/secret.html → Should be blocked/403

# View real-time AppArmor logs while testing
```

```
sudo journalctl -f | grep apparmor &
```

```
# Or monitor syslog
```

```
sudo tail -f /var/log/syslog | grep -i "apparmor"
```

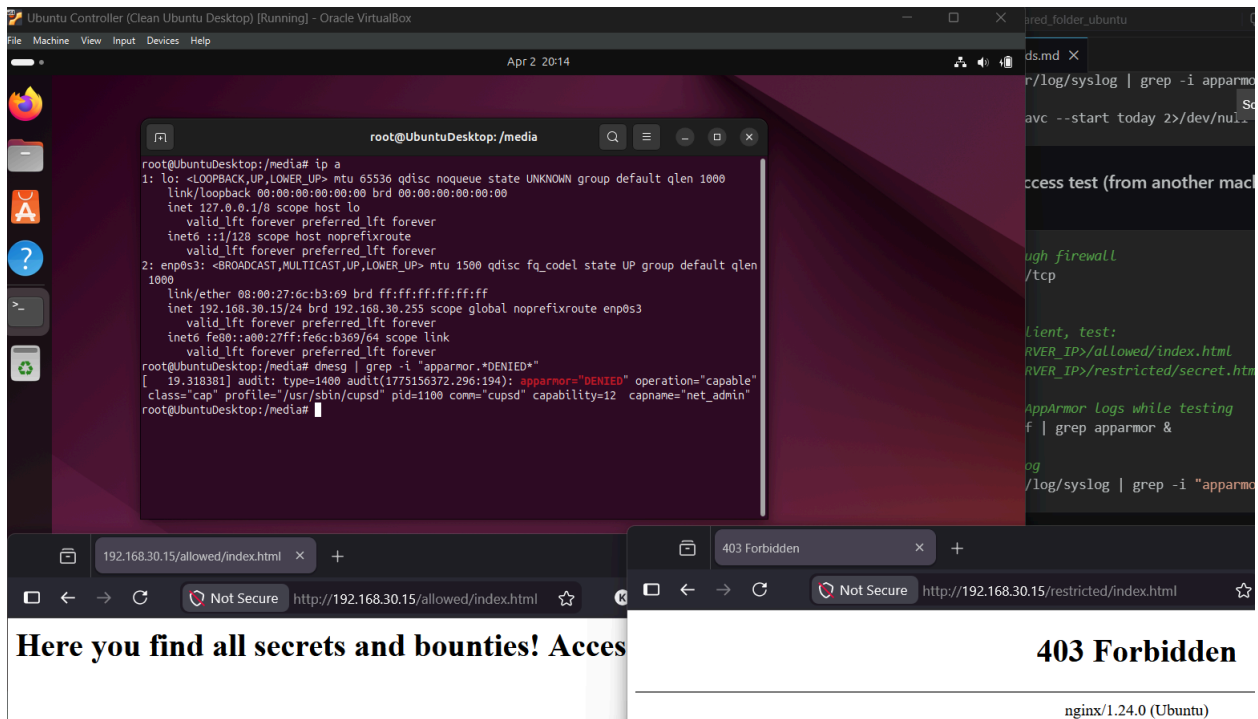


Figure. External access to web directories (AppArmor applied)

Step 10 – Temporarily disable AppArmor to confirm both work again

```
# Put profile in complain mode (logging only, no blocking)
sudo aa-complain /usr/sbin/nginx
```

```
sudo systemctl restart nginx
```

```
# Now BOTH should succeed
```

```
curl http://localhost/allowed/index.html
```

```
curl http://localhost/restricted/secret.html
```

```
# Re-enforce
sudo aa-enforce /usr/sbin/nginx
sudo systemctl restart nginx
```

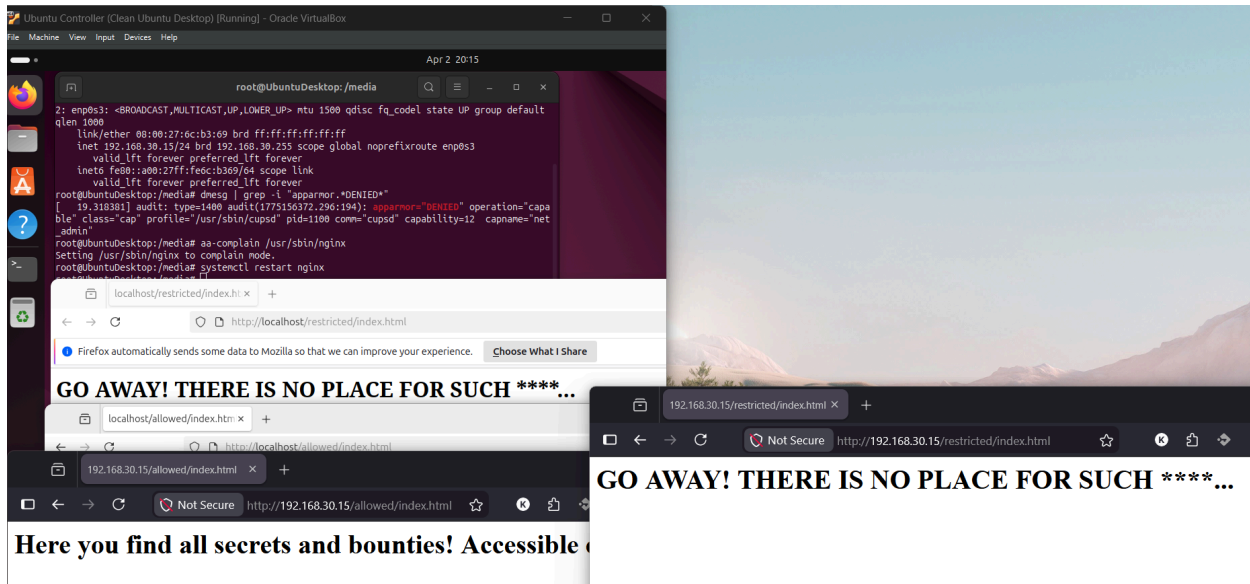


Figure. External access to web directories (AppArmor disabled)

A4 – How AppArmor Uses Default Profiles to Secure Services

- Each process/system (`usr/sbin/apache2`) has it's own default profile inside apparmor etc directory (`/etc/apparmor.d/`) which defines files, directories, network access, system calls and their access policies respectively. Works at the top of UNIX chown/chmod rules. Not defined routes and accesses are blocked.
- List active profiles `sudo aa-status` . Mode profiles: enforce (block undefined actions), complain (allow but log violations).
- Add custom rule:

```
nano /etc/apparmor.d/usr.bin.httpd
# add line
/srv/data/** r, # read inside this directory all files
```

```
and stuff recursively
```

```
sudo systemctl restart apparmor
```

List of useful commands:

```
# List all pre-installed default profiles
ls /etc/apparmor.d/
```

```
# View a common default profile (e.g., cups)
sudo cat /etc/apparmor.d/usr.sbin.cupsd
```

```
# View abstractions that profiles include
ls /etc/apparmor.d/abstractions/
```

```
# Example: see what the 'base' abstraction permits
sudo cat /etc/apparmor.d/abstractions/base
```

```
# See which processes are currently confined
sudo aa-status
```

```
# See processes WITHOUT a profile (unconfined)
sudo aa-status | grep "unconfined"
```

```
# Check profile for a specific binary
sudo apparmor_parser -p /etc/apparmor.d/usr.sbin.nginx
```

```
# Display profile details
sudo aa-status --json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for mode, profs in data.get('profiles', {}).items():
    print(f'Mode: {mode}')
    for p in profs[:5]:
```

```
        print(f' {p}')
```

```
    "
```

A5 – Troubleshooting AppArmor Confinement Issues

When your Webapp fails to start or misbehaves after AppArmor enforcement:

Step 1 – Identify the problem

```
# Check service status
sudo systemctl status nginx

# Check AppArmor logs for denials
sudo dmesg | grep apparmor | tail -30
sudo grep -i apparmor /var/log/syslog | tail -30
sudo grep -i "DENIED" /var/log/kern.log | tail -30

# If auditd is installed
sudo ausearch -m avc --start recent
```

Step 2 – Switch to Complain mode (non-blocking logging)

```
# Complain mode: log denials but don't block
sudo aa-complain /usr/sbin/nginx
sudo systemctl restart nginx

# Now reproduce the issue
curl http://localhost/restricted/secret.html

# Read what was denied/allowed
sudo aa-logprof # Interactive: reviews logs and suggests pr
ofile updates
# OR
sudo cat /var/log/syslog | grep apparmor
```

Step 3 – Use aa-logprof to auto-update the profile

```
# After running in complain mode, run aa-logprof to analyse
sudo aa-logprof

# It will interactively show denied operations and suggest ad
ditions
# Press 'A' to allow, 'D' to deny, 'S' to save

# After updating, re-enforce
sudo aa-enforce /usr/sbin/nginx
sudo systemctl restart nginx
```

Step 4 – Manually edit the profile to add missing permissions

```
# Edit profile
sudo vim /etc/apparmor.d/usr.sbin.nginx

# Example fix: if nginx needs /var/www/uploads/
# Add inside the profile block:
#   /var/www/uploads/ r,
#   /var/www/uploads/** rw,

# After editing, reload the profile
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.nginx

# Restart the service
sudo systemctl restart nginx

# Verify no more denials
sudo dmesg | grep apparmor | tail -10
```

Step 5 – Use aa-genprof to generate a fresh profile

```
# Generate a brand-new profile for a binary
sudo aa-genprof /usr/sbin/nginx

# In another terminal, exercise the application:
#   curl http://localhost/allowed/index.html
#   curl http://localhost/restricted/secret.html
# Then come back and press 'S' to scan logs and 'F' to finish

# The new profile will be in complain mode; enforce when satisfied
sudo aa-enforce /usr/sbin/nginx
```

Step 6 – Disable AppArmor for one profile only (last resort)

```
# Disable just nginx profile without disabling AppArmor system-wide
sudo aa-disable /usr/sbin/nginx

# Or move to a safe backup
sudo mv /etc/apparmor.d/usr.sbin.nginx /etc/apparmor.d/usr.sbin.nginx.bak

# Re-enable later
sudo mv /etc/apparmor.d/usr.sbin.nginx.bak /etc/apparmor.d/usr.sbin.nginx
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.nginx
sudo aa-enforce /usr/sbin/nginx
```

PART B – SELinux

B1 – SELinux Explained (Conceptual Summary)

SELinux (Security-Enhanced Linux) is a MAC framework developed by the NSA, integrated into the Linux kernel. It uses policy-based access control where every process, file, and socket is labelled with a security context. Actions are only permitted if the SELinux policy explicitly allows them — default is DENY.

Key concepts:

- **Labels / Contexts:** `user:role:type:level` (e.g., `system_u:system_r:httpd_t:s0`)
 - `type` = classification (e.g., `user_home_t`)
 - `level` = sensitivity (e.g., `s0`)
 - **Modes:** Enforcing (blocks + logs), Permissive (logs only), Disabled
 - **Policy types:** Targeted (most services), MLS (multi-level security)
 - **Boolean switches:** Toggle permissions without rewriting policy
-

B2 – Deploy Webapp, Stress Test, Install SELinux, Compare

Phase 1 – Deploy the Webapp (Python Flask on Ubuntu 22.04)

```
# Install Python and pip
sudo apt install -y python3 python3-pip python3-venv

# Create app directory
sudo mkdir -p /opt/webapp
cd /opt/webapp

# Create virtual environment
python3 -m venv venv
source venv/bin/activate

# Install Flask
pip install flask gunicorn
```

```

# Create the Flask app
cat > /opt/webapp/app.py << 'EOF'
from flask import Flask, jsonify
import time, os, socket

app = Flask(__name__)

@app.route('/')
def index():
    return jsonify({
        "status": "running",
        "host": socket.gethostname(),
        "pid": os.getpid()
    })

@app.route('/health')
def health():
    return jsonify({"healthy": True})

@app.route('/compute')
def compute():
    # Light CPU work
    result = sum(i*i for i in range(100000))
    return jsonify({"result": result})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
EOF

# Create systemd service for the webapp
sudo tee /etc/systemd/system/webapp.service << 'EOF'
[Unit]
Description=Lab5 Flask Webapp
After=network.target

[Service]

```

```
User=www-data
Group=www-data
WorkingDirectory=/opt/webapp
ExecStart=/opt/webapp/venv/bin/gunicorn --workers 4 --bind 0.0.0.0:5000 app:app
Restart=always
RestartSec=3
```

```
[Install]
WantedBy=multi-user.target
EOF
```

```
# Fix permissions
sudo chown -R www-data:www-data /opt/webapp
```

```
sudo systemctl daemon-reload
sudo systemctl enable webapp
sudo systemctl start webapp
sudo systemctl status webapp
```

```
# Test
curl http://localhost:5000/
curl http://localhost:5000/health
curl http://localhost:5000/compute
```

```
(venv) root@UbuntuDesktop:/opt/webapp# systemctl status webapp
● webapp.service - Lab5 Flask Webapp
   Loaded: loaded (/etc/systemd/system/webapp.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-04-02 20:32:50 UTC; 45s ago
     Main PID: 21028 (gunicorn)
        Tasks: 5 (limit: 7159)
      Memory: 72.2M (peak: 72.7M)
         CPU: 408ms
       CGroup: /system.slice/webapp.service
               └─21028 /opt/webapp/venv/bin/python3 /opt/webapp/venv/bin/gunicorn --wo
                 └─21029 /opt/webapp/venv/bin/python3 /opt/webapp/venv/bin/gunicorn --wo
                 └─21030 /opt/webapp/venv/bin/python3 /opt/webapp/venv/bin/gunicorn --wo
```

Figure. Webapp is running

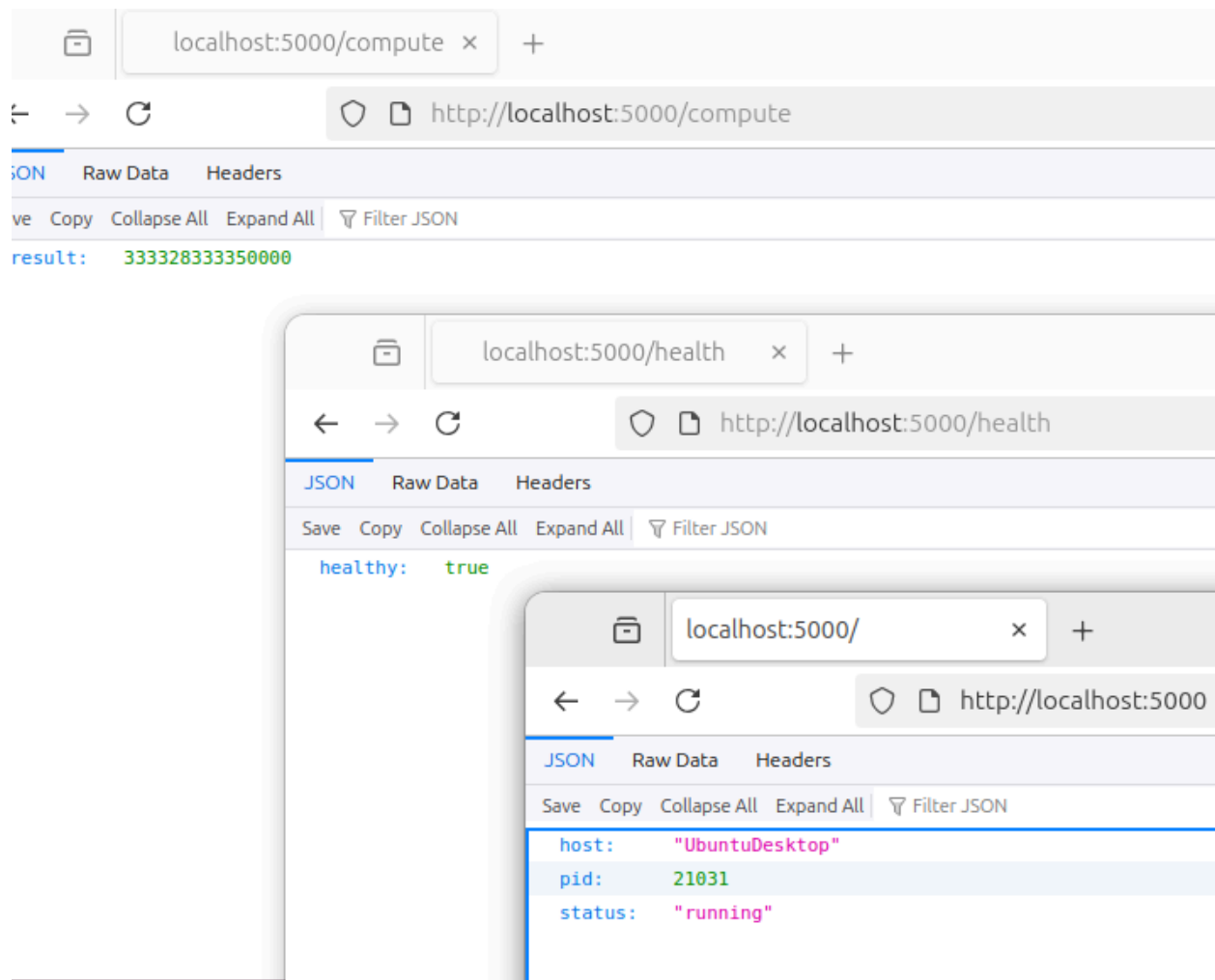


Figure. Webapp is accessible

Phase 2 – Stress Test BEFORE SELinux (Baseline)

```
# Install stress testing tools
sudo apt install -y apache2-utils wrk stress sysstat

# Allow port 5000
sudo ufw allow 5000/tcp

# ---- Benchmark 1: Apache Bench (ab) ----
echo "=== Apache Bench Baseline (no SELinux) ==="
```

```

ab -n 5000 -c 50 -r http://localhost:5000/ 2>&1 | tee /tmp/ba
seline_ab.txt

# Show key results
grep -E "Requests per second|Time per request|Failed request
s" /tmp/baseline_ab.txt

# ---- Benchmark 2: wrk (detailed throughput) ----
echo "=== wrk Baseline ==="
wrk -t4 -c50 -d30s http://localhost:5000/ 2>&1 | tee /tmp/bas
eline_wrk.txt

# ---- Benchmark 3: CPU stress + load monitoring ----
echo "=== System resource usage during load ==="
# Start stress in background
stress --cpu 2 --timeout 30 &

# Monitor for 30 seconds
for i in $(seq 1 6); do
    echo "--- Sample$i ---"
    mpstat 1 1
    sleep 5
done

# ---- Benchmark 4: Response time measurement ----
echo "=== Response time baseline ==="
for i in $(seq 1 10); do
    curl -o /dev/null -s -w "%{time_total}\n" http://localhos
t:5000/compute
done | awk '{sum+=$1; count++;} END {printf "Average response
time: %.4f seconds\n", sum/count}'

# Save baseline summary
cat > /tmp/baseline_summary.txt << EOF
=== BASELINE (No SELinux) ===
Date:$(date)

```

```
$(grep -E "Requests per second|Time per request|Failed" /tmp/  
baseline_ab.txt)  
$(cat /tmp/baseline_wrk.txt | tail -5)  
EOF
```

```
cat /tmp/baseline_summary.txt
```

```
(venv) root@UbuntuDesktop:/opt/webapp# ufw allow 5000/tcp  
Rule added  
Rule added (v6)  
(venv) root@UbuntuDesktop:/opt/webapp# ab -n 5000 -c 50 -r http://localhost:5000/ 2>&1 | tee /tmp/bas  
eline_ab.txt  
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>  
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/  
Licensed to The Apache Software Foundation, http://www.apache.org/  
  
Benchmarking localhost (be patient)  
Completed 500 requests  
Completed 1000 requests  
Completed 1500 requests  
Completed 2000 requests  
Completed 2500 requests  
Completed 3000 requests  
Completed 3500 requests  
Completed 4000 requests  
Completed 4500 requests  
Completed 5000 requests  
Finished 5000 requests  
  
Server Software:      gunicorn  
Server Hostname:      localhost  
Server Port:          5000
```

```

- Complete requests:      5000
- Failed requests:        1
  (Connect: 0, Receive: 1, Length: 0, Exceptions: 0)
Total transferred:      1004799 bytes
HTML transferred:       279944 bytes
Requests per second:    2497.11 [#/sec] (mean)
Time per request:       20.023 [ms] (mean)
Time per request:       0.400 [ms] (mean, across all concurrent requests)
Transfer rate:          490.06 [Kbytes/sec] received

Connection Times (ms)
      min    mean[+/-sd] median    max
Connect:    0      0   0.6      0      5
Processing:  0     20   6.0     18     75
Waiting:    0     19   5.7     17     75
Total:      0     20   5.8     18     75

Percentage of the requests served within a certain time (ms)
 50%    18
 66%    21
 75%    23
 80%    24
 90%    28
 95%    31
 98%    36
 99%    39
100%    75 (longest request)
(venv) root@UbuntuDesktop:/opt/webapp# grep -E "Requests|Time per|Failed" /tmp/baseline_ab.txt

```

```

(venv) root@UbuntuDesktop:/opt/webapp# grep -E "Requests|Time per|Failed" /tmp/baseline_ab.txt
Failed requests:      1
Requests per second:  2497.11 [#/sec] (mean)
Time per request:     20.023 [ms] (mean)
Time per request:     0.400 [ms] (mean, across all concurrent requests)
(venv) root@UbuntuDesktop:/opt/webapp#

```

Figure. Benchmark 1 results

```

(venv) root@UbuntuDesktop:/opt/webapp# wrk -t4 -c50 -d30s http://localhost:5000/ 2>&1 | tee /tmp_baseline_wrk.txt
Running 30s test @ http://localhost:5000/
4 threads and 50 connections
  Thread Stats   Avg      Stdev     Max   +/-  Stdev
  Latency    17.33ms    4.35ms  115.54ms   87.44%
  Req/Sec   695.27    116.55    848.00   75.56%
83184 requests in 30.10s, 15.95MB read
Requests/sec:  2763.93
Transfer/sec:   542.53KB
(venv) root@UbuntuDesktop:/opt/webapp#

```

Figure. Benchmark 2 results

```
^C
(venv) root@UbuntuDesktop:/opt/webapp# stress --cpu 2 --timeout 30 & stress: info: [24612] successful
run completed in 31s
stress: info: [24629] successful run completed in 30s
```

Figure. Benchmark 3 results

```
root@UbuntuDesktop:/media#
(venv) root@UbuntuDesktop:/media# for i in $(seq 1 10); do
    curl -o /dev/null -s -w "%{time_total}\n" http://localhost:5000/compute
done | awk '{sum+=$1; count++;} END {printf "Average reposncse time: %.4f seconds\n", sum/count}'
Average reposncse time: 0.0067 seconds
(venv) root@UbuntuDesktop:/media#
```

Figure. Benchmark 4 results

Phase 3 – Install SELinux on Ubuntu 22.04

Note: Ubuntu uses AppArmor by default. SELinux requires disabling AppArmor first.

```
# Step 1: Disable AppArmor (SELinux will replace it)
sudo systemctl stop apparmor
sudo systemctl disable apparmor
sudo apt remove -y --purge apparmor apparmor-utils
sudo apt autoremove -y

# Step 2: Install SELinux packages
sudo apt install -y selinux-basics selinux-policy-default aud
itd

# Step 3: Activate SELinux (sets up filesystem labels)
sudo selinux-activate
# This adds security=selinux to /etc/default/grub and updates
GRUB

# Step 4: Verify the activation
```



```
grep "selinux" /etc/default/grub
cat /etc/selinux/config
```

```
# Step 5: REQUIRED REBOOT – first boot will relabel the entire filesystem
echo "Rebooting to apply SELinux and relabel filesystem..."
sudo reboot
```

```
root@UbuntuDesktop:~# systemctl stop apparmor
root@UbuntuDesktop:~# systemctl disable apparmor
Synchronizing state of apparmor.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install disable apparmor
Removed "/etc/systemd/system/sysinit.target.wants/apparmor.service".
root@UbuntuDesktop:~# apt remove -y --purge apparmor apparmor-utils
Reading package lists... Done
Building dependency tree... Done
```

Figure. Reinstalling AppArmor

```
root@UbuntuDesktop:~# selinux-activate
Activating SE Linux
Sourcing file '/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-20-generic
Found initrd image: /boot/initrd.img-6.17.0-20-generic
Found linux image: /boot/vmlinuz-6.14.0-37-generic
Found initrd image: /boot/initrd.img-6.14.0-37-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
SE Linux is activated. You may need to reboot now.
root@UbuntuDesktop:~# grep "selinux" /etc/default/grub
GRUB_CMDLINE_LINUX=" apparmor=1 security=apparmor security=selinux"
```

Figure. SELINUX is installed and active

After first reboot – Verify SELinux status

```

# Check SELinux status
sestatus

# Should show:
# SELinux status:                enabled
# SELinuxfs mount:              /sys/fs/selinux
# SELinux mount check:          enabled
# Mount check:                   enabled
# Rootfs labeled:               disabled
# Loaded policy name:           default
# Current mode:                  permissive  <-- starts per
missive
# Mode from config file:         permissive
# Policy MLS status:             enabled
# Policy deny_unknown status:    denied
# Memory protection checking:    actual (secure)
# Max kernel policy version:     33

# Check mode
getenforce
# Output: Permissive

# List contexts
ls -Z /var/www/
ls -Z /opt/webapp/

```

```

root@UbuntuDesktop:~# sestatus
SELinux status:                enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:        /etc/selinux
Loaded policy name:              default
Current mode:                   permissive
Mode from config file:          permissive
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      35
root@UbuntuDesktop:~# getenforce
Permissive
root@UbuntuDesktop:~# ls -Z /var/www/
system_u:object_r:httpd_sys_content_t:s0 allowed
system_u:object_r:httpd_sys_content_t:s0 html
system_u:object_r:httpd_sys_content_t:s0 restricted
root@UbuntuDesktop:~# ls -z /opt/webapp/
ls: invalid option -- 'z'
Try 'ls --help' for more information.
root@UbuntuDesktop:~# ls -Z /opt/webapp/
system_u:object_r:usr_t:s0 app.py          system_u:object_r:usr_t:s0 venv
system_u:object_r:usr_t:s0 __pycache__
root@UbuntuDesktop:~#

```

Figure. SELinux status

Phase 4 – Configure SELinux for the Webapp

```

# Install SELinux utilities
sudo apt install -y policycoreutils setools python3-setools

# Restart webapp after reboot
sudo systemctl start webapp

# Check context of the webapp files
ls -Z /opt/webapp/

# Check current process context
ps -eZ | grep gunicorn

```

```
# Allow webapp to bind to port 5000
# First check if port is already defined
sudo semanage port -l | grep 5000

# Add port 5000 to http_port_t
sudo semanage port -a -t http_port_t -p tcp 5000 2>/dev/null
|| \
    sudo semanage port -m -t http_port_t -p tcp 5000

# Verify
sudo semanage port -l | grep http_port_t

# Label the webapp directory appropriately
sudo semanage fcontext -a -t httpd_exec_t '/opt/webapp/venv/bin/gunicorn'
sudo semanage fcontext -a -t httpd_sys_content_t '/opt/webapp(/.*)?'
sudo restorecon -Rv /opt/webapp/

# Verify new labels
ls -Z /opt/webapp/
```

```

root@UbuntuDesktop:~# ps -eZ | grep gunicorn
system_u:system_r:initrc_t:s0      745 ?          00:00:00 gunicorn
system_u:system_r:initrc_t:s0      778 ?          00:00:00 gunicorn
system_u:system_r:initrc_t:s0      788 ?          00:00:00 gunicorn
system_u:system_r:initrc_t:s0      789 ?          00:00:00 gunicorn
system_u:system_r:initrc_t:s0      791 ?          00:00:00 gunicorn
root@UbuntuDesktop:~# semanage port -l | grep 5000
cluster_port_t      tcp      5149, 40040, 50006-50008
cluster_port_t      udp      5149, 50006-50008
complex_main_port_t tcp      5000
complex_main_port_t udp      5000
hplip_port_t        tcp      1782, 2207, 2208, 8290, 8292, 9100, 9101
, 9102, 9220, 9221, 9222, 9280, 9281, 9282, 9290, 9291, 50000, 50002
root@UbuntuDesktop:~# semanage port -a -t http_port_t -p tcp 5000 2>/dev/null ||
sudo semanage port -m -t http_port_t -p tcp 5000
libsemanage.get_home_dirs: Error while fetching users. Returning list so far.
libsemanage.add_user: user sddm not in password file
root@UbuntuDesktop:~# semanage port -l | grep http_port_t
http_port_t         tcp      5000, 80, 443, 488, 8008, 8009, 8443, 84
48

```

Figure. gunicorn server status

```

root@UbuntuDesktop:~# semanage fcontext -a -t httpd_sys_content_t '/opt/webapp(/
.*)?'
libsemanage.get_home_dirs: Error while fetching users. Returning list so far.
libsemanage.add_user: user sddm not in password file
root@UbuntuDesktop:~# semanage fcontext -a -t httpd_exec_t '/opt/webapp/venv/bin
/gunicorn'
ValueError: File context for /opt/webapp/venv/bin/gunicorn already defined
root@UbuntuDesktop:~# restorecon -Rv /opt/webapp/
Relabeled /opt/webapp from system_u:object_r:usr_t:s0 to system_u:object_r:httpd
_sys_content_t:s0

```

Figure. Labeling webapp directories

Phase 5 – Implement SELinux Containment Policies

```

# Check for SELinux denials so far
sudo ausearch -m avc --start today 2>/dev/null | head -40

# ---- Policy 1: Use audit2allow to create a custom policy mo
dule ----

```

```

# Run webapp in permissive mode and capture denials
sudo setenforce 0
sudo systemctl restart webapp
curl http://localhost:5000/compute
curl http://localhost:5000/

# Collect AVC denials and create a policy
sudo ausearch -m avc --start today 2>/dev/null | \
    audit2allow -M webapp_policy 2>/dev/null || echo "No denials
to convert"

# If a policy was created, install it
if [ -f webapp_policy.pp ]; then
    sudo semodule -i webapp_policy.pp
    echo "Custom policy module installed"
fi

# ---- Policy 2: SELinux Booleans ----

# List booleans related to httpd (web)
sudo semanage boolean -l | grep httpd | head -20

# Allow httpd to connect to network (needed for gunicorn)
sudo setsebool -P httpd_can_network_connect 1

# Allow httpd to read user content
sudo setsebool -P httpd_read_user_content 1

# Check active booleans
getsebool -a | grep httpd | grep "on$"

# ---- Policy 3: Restrict file access ----
# Create a sensitive file and label it so the webapp CANNOT read it
sudo mkdir -p /opt/secrets
echo "TOP SECRET DATA" | sudo tee /opt/secrets/secret.txt

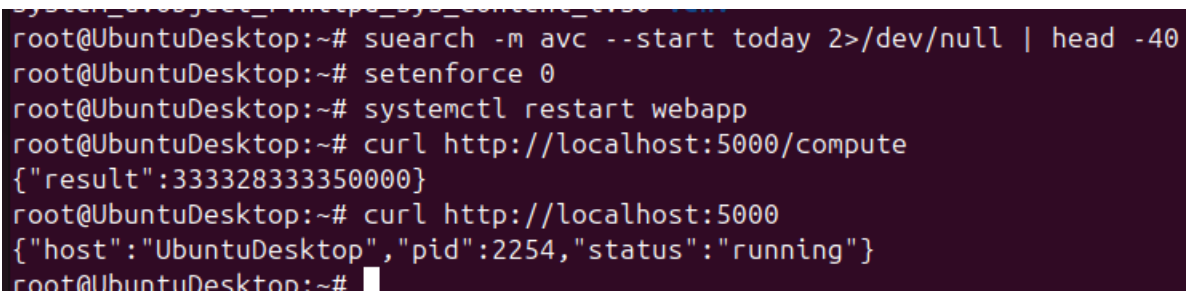
```

```
# Label with a type that httpd cannot access
sudo semanage fcontext -a -t etc_t '/opt/secrets(/.*)?'
sudo restorecon -Rv /opt/secrets/
ls -Z /opt/secrets/

# Now switch to Enforcing mode
sudo setenforce 1
getenforce
# Should output: Enforcing

# ---- Policy 4: Enable SELinux enforcing in config (permanen
t) ----
sudo sed -i 's/SELINUX=permissive/SELINUX=enforcing/' /etc/se
linux/config
grep SELINUX /etc/selinux/config

# Restart webapp and verify it still runs
sudo systemctl restart webapp
sudo systemctl status webapp
curl http://localhost:5000/health
```



```
root@UbuntuDesktop:~# sestatus -m avc --start today 2>/dev/null | head -40
root@UbuntuDesktop:~# setenforce 0
root@UbuntuDesktop:~# systemctl restart webapp
root@UbuntuDesktop:~# curl http://localhost:5000/compute
{"result":333328333350000}
root@UbuntuDesktop:~# curl http://localhost:5000
{"host":"UbuntuDesktop","pid":2254,"status":"running"}
root@UbuntuDesktop:~#
```

Figure. Webapp is running in permissive mode

```

root@UbuntuDesktop:~# ausearch -m avc --start today 2>/dev/null | audit2allow -
m webapp_policy 2>/dev/null || echo "NO DENIAL TO CONVERT"

module webapp_policy 1.0;

require {
    type user_home_dir_t;
    type xdg_data_t;
    type kmod_t;
    type systemd_logind_t;
    type init_runtime_t;
    type security_t;
    type etc_runtime_t;
    type udev_runtime_t;
    type NetworkManager_t;

```

```

root@UbuntuDesktop:~# cat tx.sh
if [ -f webapp_policy.pp ]; then
    sudo semodule -i webapp_policy.pp
    echo "Policy module is installed"
fi

```

Figure. Policy 1 is created

```

root@UbuntuDesktop:~# semanage boolean -l | grep httpd | head -20
allow_httpd_anon_write (off , off) Determine whether httpd can modify
public files used for public file transfer services. Directories/Files must be
labeled public_content_rw_t.
allow_httpd_apcupsd CGI_script_anon_write (off , off) Determine whether the s
cript domain can modify public files used for public file transfer services. Dir
ectories/Files must be labeled public_content_rw_t.
allow_httpd_awstats_script_anon_write (off , off) Determine whether the scrip
t domain can modify public files used for public file transfer services. Directo
ries/Files must be labeled public_content_rw_t.
allow_httpd_collectd_script_anon_write (off , off) Determine whether the scri
pt domain can modify public files used for public file transfer services. Direct
ories/Files must be labeled public_content_rw_t.

```

```

root@UbuntuDesktop:~# setsebool -P httpd_read_user_content 1
root@UbuntuDesktop:~# setsebool -P httpd_can_network_connect 1
root@UbuntuDesktop:~# getsebool -a | grep httpd | grep "on$"
httpd_can_network_connect --> on
httpd_read_user_content --> on
root@UbuntuDesktop:~#

```


Figure. Policy 2 is created

```
GNU nano 7.2 /opt/secrets/secret.txt *
TOP TOP TOP SECRET!!!

File Name to Write: /opt/secrets/secret.txt

root@UbuntuDesktop:~# semanage fcontext -a -t etc_t '/opt/secrets(/.*)?'
libsemanage.get_home_dirs: Error while fetching users. Returning list so far.
libsemanage.add_user: user sddm not in password file
root@UbuntuDesktop:~# restorecon -Rv /opt/secrets/
Relabeled /opt/secrets from unconfined_u:object_r:usr_t:s0 to unconfined_u:object_r:etc_t:s0
Relabeled /opt/secrets/secret.txtY from unconfined_u:object_r:usr_t:s0 to unconfined_u:object_r:etc_t:s0
root@UbuntuDesktop:~# ls -Z /opt/secrets/
unconfined_u:object_r:etc_t:s0 secret.txtY
root@UbuntuDesktop:~#

root@UbuntuDesktop:~# setenforce 1
root@UbuntuDesktop:~# getenforce
Enforcing
root@UbuntuDesktop:~#
```

Figure. Policy 3 is created

```
root@UbuntuDesktop:~# sed -i 's/SELINUX=permissive/SELINUX=enforcing/' /etc/selinux/config
root@UbuntuDesktop:~# grep SELINUX /etc/selinux/config
# SELINUX= can take one of these three values:
SELINUX=enforcing
# SELINUXTYPE= can take one of these two values:
SELINUXTYPE=default
```

```
SELINUXTYPE=default
root@UbuntuDesktop:~# systemctl restart webapp
root@UbuntuDesktop:~# systemctl status webapp
● webapp.service - Lab5 Flask Webapp
   Loaded: loaded (/etc/systemd/system/webapp.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-04-03 09:26:11 UTC; 5s ago
     Main PID: 2190 (gunicorn)
        Tasks: 5 (limit: 7159)
      Memory: 69.5M (peak: 69.7M)
         CPU: 402ms
       CGroup: /system.slice/webapp.service
               └─2190 /opt/webapp/venv/bin/python3 /opt/webapp/venv/bin/gunicorn >
                 └─2191 /opt/webapp/venv/bin/python3 /opt/webapp/venv/bin/gunicorn >
                   └─2192 /opt/webapp/venv/bin/python3 /opt/webapp/venv/bin/gunicorn >
```

```
Apr 03 09:26:11 UbuntuDesktop gunicorn[2190]: [2026-04-03 09:26:11]
root@UbuntuDesktop:~# curl http://localhost:5000
{"host": "UbuntuDesktop", "pid": 2194, "status": "running"}
root@UbuntuDesktop:~# curl http://localhost:5000/health
{"healthy": true}
root@UbuntuDesktop:~#
```

Figure. Policy 4 is created

Phase 6 – Stress Test AFTER SELinux (Enforcing)

```
# Confirm SELinux is enforcing
getenforce

# ---- Benchmark 1: Apache Bench with SELinux ----
echo "=== Apache Bench WITH SELinux Enforcing ==="
```

```
ab -n 5000 -c 50 -r http://localhost:5000/ 2>&1 | tee /tmp/selinux_ab.txt
```

```
grep -E "Requests per second|Time per request|Failed requests" /tmp/selinux_ab.txt
```

```
# ---- Benchmark 2: wrk with SELinux ----
```

```
echo "=== wrk WITH SELinux ==="
```

```
wrk -t4 -c50 -d30s http://localhost:5000/ 2>&1 | tee /tmp/selinux_wrk.txt
```

```
# ---- Benchmark 3: Response time ----
```

```
echo "=== Response time WITH SELinux ==="
```

```
for i in $(seq 1 10); do
```

```
    curl -o /dev/null -s -w "%{time_total}\n" http://localhost:5000/compute
```

```
done | awk '{sum+=$1; count++} END {printf "Average response time: %.4f seconds\n", sum/count}'
```

```
# ---- Compare Results ----
```

```
echo ""
```

```
echo "=====
```

```
echo "          PERFORMANCE COMPARISON SUMMARY"
```

```
echo "=====
```

```
echo ""
```

```
echo "--- BASELINE (No SELinux) ---"
```

```
grep -E "Requests per second|Time per request|Failed" /tmp/baseline_ab.txt 2>/dev/null
```

```
echo ""
```

```
echo "--- WITH SELinux Enforcing ---"
```

```
grep -E "Requests per second|Time per request|Failed" /tmp/selinux_ab.txt 2>/dev/null
```

```
echo ""
```

```
echo "Note: Performance difference is typically < 1-3% overhead"
```

```
ad."
echo "SELinux kernel-level enforcement has minimal throughput
impact."
echo "=====
```

BONUS – Shellshock Attack & SELinux Mitigation

Background

Shellshock (CVE-2014-6271) exploits a Bash bug where function definitions in environment

variables are followed by arbitrary commands that execute when Bash starts.

```
env VAR='() { :; }; <COMMAND>' bash
```

CGI scripts that invoke Bash are vulnerable because web servers pass HTTP headers

as environment variables to CGI scripts.

Step 1 – Set up the vulnerable environment

```
# Install an old Bash version in a test directory (simulate the vulnerability)
# On modern Ubuntu, Bash is already patched. We simulate using a custom CGI setup.
```

```
# Install Apache with CGI support
sudo apt install -y apache2
```

```
# Enable CGI module
sudo a2enmod cgi
sudo a2enconf serve-cgi-bin
sudo systemctl restart apache2
```

```
# Create a vulnerable CGI script (simulates calling bash with
```

```

env vars)
sudo mkdir -p /usr/lib/cgi-bin/
sudo tee /usr/lib/cgi-bin/test.sh << 'CGISCRIP
#!/bin/bash
echo "Content-type: text/plain"
echo ""
echo "CGI script executed by: $(whoami)"
echo "Hostname: $(hostname)"
CGISCRIP

sudo chmod +x /usr/lib/cgi-bin/test.sh
sudo chown www-data:www-data /usr/lib/cgi-bin/test.sh

# Test normal CGI execution
curl http://localhost/cgi-bin/test.sh

```

Step 2 – Test the Shellshock vector (on patched system, shows blocked attempt)

```

# This simulates what the Shellshock attack does:
# Attacker sends a malicious User-Agent header with bash func
tion + command

# Test 1 – Basic shellshock probe
echo "=== Shellshock Probe ==="
curl -A '() { ;; }; echo "VULNERABLE: shellshock worked"' \
http://localhost/cgi-bin/test.sh

# Test 2 – Try to read /etc/passwd via shellshock
echo "=== Password File Exfil Attempt ==="
curl -H 'X-Custom: () { ;; }; /bin/cat /etc/passwd' \
http://localhost/cgi-bin/test.sh

# Test 3 – Try to create a reverse shell
echo "=== Reverse Shell Attempt ==="

```

```

curl -A '() { ;; }; /bin/bash -c "id > /tmp/shellshock_proof.
txt"' \
    http://localhost/cgi-bin/test.sh

# Check if file was created (indicates shellshock success)
if [ -f /tmp/shellshock_proof.txt ]; then
    echo "SHELLSHOCK SUCCEEDED - file created:"
    cat /tmp/shellshock_proof.txt
else
    echo "SHELLSHOCK BLOCKED or system is patched"
fi

```

Step 3 – Demonstrate SELinux blocking Shellshock

```

# Ensure SELinux is enforcing
sudo setenforce 1
getenforce

# Label the CGI directory
ls -Z /usr/lib/cgi-bin/

# Apply the correct SELinux context for CGI scripts
sudo semanage fcontext -a -t httpd_sys_script_exec_t '/usr/li
b/cgi-bin(/.*)?'
sudo restorecon -Rv /usr/lib/cgi-bin/

# Verify context
ls -Z /usr/lib/cgi-bin/test.sh

# Try the shellshock attack again WITH SELinux enforcing
echo "=== Shellshock with SELinux Enforcing ==="

# Attempt to write to /tmp via shellshock
curl -A '() { ;; }; /bin/bash -c "id > /tmp/selinux_test.txt
2>&1"' \

```

```
http://localhost/cgi-bin/test.sh
```

```
# Check if it worked
```

```
ls -la /tmp/selinux_test.txt 2>/dev/null && \  
echo "FILE CREATED - SELinux did not block" || \  
echo "FILE NOT CREATED - SELinux blocked the attack!"
```

```
# View SELinux denial logs
```

```
sudo ausearch -m avc --start today 2>/dev/null | tail -20  
sudo dmesg | grep -i "apparmor\|selinux" | tail -20
```

```
# SELinux prevents the shellshock payload because:
```

```
# 1. httpd_t processes cannot execute /bin/bash outside polic  
y
```

```
# 2. httpd_t cannot write to /tmp (only httpd_tmp_t areas)
```

```
# 3. httpd_t cannot exec arbitrary binaries
```

```
echo ""
```

```
echo "Key SELinux containment: even if Bash IS invoked, the h  
ttpd_t domain"
```

```
echo "cannot write files outside allowed locations or spawn u  
nrestricted shells."
```

Step 4 – Demonstrate the containment clearly

```
# Check what httpd_t can and cannot do
```

```
echo "=== What httpd_t processes are NOT allowed to do: ==="
```

```
sudo sesearch --allow --source httpd_t --target shell_exec_t  
2>/dev/null | head -10
```

```
# No results = httpd cannot exec a shell
```

```
sudo sesearch --allow --source httpd_t --target tmp_t -c file  
2>/dev/null | head -10
```

```
# Restricted write access to tmp
```

```
# Verify current process context for Apache
```

```
ps -eZ | grep apache2 | head -5

# Show that the process is in a restricted domain
echo ""
echo "Apache runs in httpd_t domain – all actions are policy-
constrained."
echo "Shellshock can trigger Bash, but Bash inherits httpd_t
and cannot:"
echo "  1. Write to arbitrary filesystem paths"
echo "  2. Open network connections (unless allowed by boolea
n)"
echo "  3. Execute binaries outside policy"
echo "  4. Read sensitive files (/etc/shadow, /root/, etc.)"
```

Step 5 – SELinux audit and logging for the attack

```
# View all SELinux denials from today
sudo ausearch -m avc --start today 2>/dev/null

# Check audit log
sudo tail -30 /var/log/audit/audit.log | grep -i "denied\|av
c"

# Generate a human-readable report
sudo audit2allow -a 2>/dev/null | head -40

# Generate SELinux stats
sudo seinfo 2>/dev/null | head -20

# Final status check
echo ""
echo "=== Final SELinux Status ==="
sestatus
```



```
getenforce
sudo semodule -l | head -10
```

QUICK REFERENCE – Key Commands

```
# AppArmor
sudo aa-status                # Show all profiles and sta
tus
sudo aa-enforce /path/to/bin  # Set profile to enforce mo
de
sudo aa-complain /path/to/bin # Set profile to complain m
ode
sudo aa-disable /path/to/bin  # Disable a profile
sudo aa-genprof /path/to/bin  # Generate a new profile
sudo aa-logprof               # Update profiles from logs
sudo apparmor_parser -r profile # Reload a profile
sudo dmesg | grep apparmor    # View AppArmor kernel log

# SELinux
getenforce                    # Show current mode
sestatus                      # Detailed SELinux status
sudo setenforce 0             # Permissive mode (temporar
y)
sudo setenforce 1             # Enforcing mode (temporar
y)
ls -Z /path                   # Show file security contex
t
ps -eZ                        # Show process security con
text
sudo ausearch -m avc          # Search audit log for deni
als
sudo audit2allow -a           # Generate policy from deni
als
```

```
sudo semanage port -l          # List port labels
sudo setsebool -P boolean on/off # Set a SELinux boolean
sudo restorecon -Rv /path      # Restore file contexts
```

End of LAB5 MAC Implementation GuidePlatform: Ubuntu 22.04 LTS Server